Messaging: fifteen milliseconds of class generation per statement

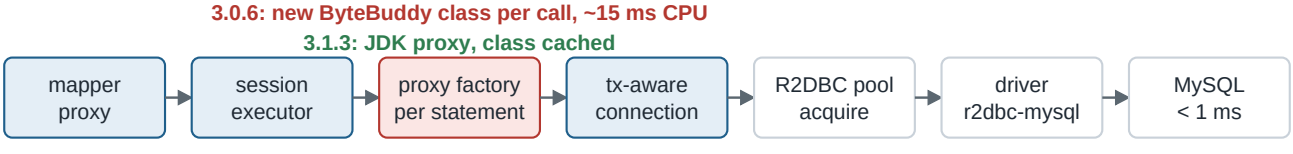
Where the cost of one statement went, how it was found, and what the fix did. Branch `feat/messaging`, third session of 11 October 2026, step 9 of the tuning order in `doc/messaging-architecture.md` section 6.

1. Summary

	Before	After
One mapper call (a point query MySQL runs in under a millisecond)	19.5 ms alone, 39.5 ms with four callers	4.2 ms alone, 4.9 ms with four callers
Why	reactive-mybatis-support 3.0.6 generated and loaded a new ByteBuddy class for every statement (two per query, one per connection acquire); about 15 ms of CPU each time and a class left behind	mybatis-r2dbc-spring 3.1.3: a JDK proxy, whose class the JDK caches per interface set
Inbox page of 20, four readers	165 ms p50, 1.4 pages/s before step 7	47 ms p50, 50 pages/s
Read-model projection, per event	39 ms	15 ms
Scope		one version property in `db-reactive/pom.xml`, so every service that uses db-reactive gets it on its next build; a test in db-reactive pins the behaviour

2. How it was found

Step 7 left a number that did not add up: MySQL executed each statement of an inbox page in well under a millisecond (EXPLAIN and profiling on the loaded rows), yet the service paid about 30 ms per statement with one reader and 80 ms with four, and probes had ruled out TLS and Nagle. The MySQL user of the service may not read the performance schema, so the measurement had to be on the client side, and layered: the same point query on one conversation row, 400 calls in sequence at each layer of the path, timed from subscription to completion as a caller sees it (`StatementCostProbeTest`, run only with `-Dprobe=true`).



measured per call, the same point query (probe layers 1 to 3):

driver alone 1.8 ms -> 2.5 ms pool 2.1 ms -> 2.9 ms mapper 19.5 ms -> 4.2 ms mapper, 4 callers 39.5 ms -> 4.9 ms

the pool and the driver did not change; everything above them did

Figure 1. The layers of one mapper call. The driver and the pool answered in one to two milliseconds; the mapper call cost 19.5 ms.

The layers below the library were innocent, and the cost stayed when all calls ran on one transaction's connection (13.5 ms per call), so it was not the connection bookkeeping either. A Flight Recorder profile of the probe (`-XX:StartFlightRecording` through surefire's `argLine`) put 769 of 924 of the CPU samples inside the library in one method, `ProxyInstanceFactory.newInstanceOfInterfaces`, under ByteBuddy's class generation. Its bytecode confirmed it: on every call, `new ByteBuddy().subclass(Object).implement(interfaces).method(isPublic()).intercept(handler).make().load(loader)`, then a constructor call; no cache, and the handler instance is baked into the generated class, so no cache was possible. The callers are the statement path (a proxy for the parameter handler, per query), the result handler (a proxy for the type-handler context, per query) and the transaction-aware connection (per acquire). That is two generated classes per query and a third per connection, each costing about five to ten milliseconds of CPU on this host and scaling with concurrency because it is CPU work.

3. The fix

The library's own release 3.1.2 of 7 July 2026 fixed exactly this ("memory leak in ProxyInstanceFactory, Byte Buddy removed", issue 178): the factory now calls `java.lang.reflect.Proxy.newProxyInstance`, and the JDK keeps one proxy class per class loader and interface set. The project's dependency was bumped from `3.0.6.RELEASE` to `3.1.3.RELEASE` (the latest 3.x, built against Spring Boot 3.5.16, MyBatis 3.5.19 and Java 17, so it fits this stack; 4.x targets Boot 4 and is not an option). The 3.1.x notes list no breaking change or renamed property; 3.1.1 added a pool customizer and an `options` map, neither of which this project uses. ByteBuddy left the compile classpath with it (it remains at test scope through Mockito).

A new test in db-reactive, `ReactiveMybatisProxyTest`, pins the behaviour: two proxies for the same interfaces come from the factory as JDK proxies of the same class. A downgrade or a regression of the library fails that test instead of showing up as latency weeks later.

4. Tests

- **db-reactive**: 70 tests, 0 failures against the live MySQL, including the new `ReactiveMybatisProxyTest`.
- **messaging**: 46 tests, 0 failures (one end-to-end timing case failed once while the db-reactive build ran beside it and passed alone) (EventSourcingDB and Kafka from Testcontainers, the read model in the live database).
- **announce-management, order, gateway-api, gateway-admin** compile against the upgraded db-reactive (`-P '!webapp' -DskipTests compile`); their own suites were not run in this session.
- `StatementCostProbeTest` is the instrument, not a test of behaviour: it prints the table of section 5 and asserts nothing.

5. Measurements

5.1 The probe, before and after

400 calls per layer, p50 (p95 in brackets), the same host, the development stack up.

Layer	3.0.6	3.1.3
driver alone, one connection, no pool	1.8 (3.2) ms	2.5 (11.8) ms
pooled factory: acquire, statement, release	2.1 (3.6) ms	2.9 (7.7) ms
mapper `selectByPrimaryKey`	19.5 (39.0) ms	4.2 (8.8) ms

Layer	3.0.6	3.1.3
mapper, 400 calls in one transaction (mean per call)	13.5 ms	2.7 ms
mapper `selectByExample`, two participant rows	13.9 (20.9) ms	3.9 (6.8) ms
pooled factory, bare `select 1`	1.0 (4.5) ms	1.2 (2.2) ms
mapper, four callers at once	39.5 (81.4) ms	4.9 (17.5) ms
pooled factory, four callers at once	1.6 (5.5) ms	3.4 (8.8) ms

The library's layer is the only one that moved. With 3.0.6 a mapper call cost 17 ms more than the pooled statement under it, 13.5 ms even on a connection it did not have to acquire, and the gap grew with concurrency because the work was CPU; with 3.1.3 the gap is about a millisecond and four callers cost what one costs. The remaining millisecond is the reactive session, the parameter and result mapping and the transaction-aware wrapper.

5.2 The service under load

The two load shapes of the earlier sections, after the upgrade, next to this afternoon's numbers with 3.0.6 (one node on 8098, a fresh throwaway store, the live MySQL; same host, same drivers).

	3.0.6 (sections 7d and 7e)	3.1.3
inbox page of 20, four readers, p50 / p95	165 / 222 ms	47 / 138 ms
inbox page of 50	182 / 257 ms	77 / 210 ms
pages served per second	20.1	49.6
`messaging.query.latency{inbox}` p50, server side	180 ms	67 ms
chat load, 800 posts: acknowledged / errors	800 / 0	800 / 0
ack latency p50 / p95 (driver)	52 / 117 ms	68 / 194 ms
`messaging.command.latency {PostMessage}` p50	47 ms	70 ms
`messaging.projection.handler {conversation-read-model}` p50, per event	39 ms	15 ms
`messaging.projection.delay {conversation-read-model}` p50 / p95	80 / 110 ms	57 / 86 ms
`messaging.live.latency{message}` p50 / p95	43 / 95 ms	51 / 167 ms

The read side is where the statements are: the read-model handler writes two tables and the checkpoint per event and now costs 15 ms instead of 39, and the projection's lag behind the command side fell with it. The inbox, two statements per page, serves two and a half times as many pages per second. The command path does not touch MySQL for a post, so its numbers are run-to-run noise (this run started while the end-to-end suite's containers were still shutting down); the ack latency is the command side's, and step 8 is about it.

6. Operating notes

- **Every service pays this.** The projections of announce-management and order, the edge controllers of the gateways and every read model query went through the same factory. Their numbers were not measured here; expect the same shape (a few milliseconds per statement less, and much less under concurrency).
- **Shared `~/.m2`.** Another worktree that installs db-reactive from an older branch puts 3.0.6 back for everyone until this branch is merged and reinstalled; `ReactiveMybatisProxyTest` is the tell.
- **What remains per statement.** About 1.3 ms for a mapper call against 2.9 ms for the pool and driver: the reactive session, the result mapping and the transaction-aware wrapper. Worth a profile only if a read model ever needs more than a few hundred statements per second per node.
- **Step 8 (command-side throughput)** is the next open item; the command path does not touch MySQL for a post, so this fix does not change it, but the read model that trails it now costs a third per event.

7. Files changed

File	Change
db-reactive/pom.xml	`mybatis-r2dbc.version` 3.0.6.RELEASE -> 3.1.3.RELEASE
db-reactive: test ReactiveMybatisProxyTest.java	new: pins the JDK proxy and the cached proxy class
messaging: test StatementCostProbeTest.java	new: the layered probe, on request only
doc/messaging-architecture.md, doc/messaging-context.md	sections 2.8, 6 (step 9 done) and the new 7f; context section 7

Branch state: the upgrade, the two tests and the doc are commit `72db52ac` on `feat/messaging`; this report and the chat-load figures of section 7f follow in the next commit.